

# Artificial Intelligence

## Assignment 3



### Topics:

Minimax,  $\alpha\beta$ -pruning on Chess Env

### Team Members:

M Akash - CS22B037

Akshatha R H - CS22B003

# Minimax Algorithm on Chess env

## Environment Setup:

- Built using gym + python-chess
- Minimax algorithm implemented for both players

## Minimax Search Logic:

- Searches moves up to a fixed depth (e.g., 3)
- Chooses move with best heuristic score for the player
- Executes alternating turns for both sides (White & Black)

## Evaluation function based on:

- Piece values:  =1,   =3,  =5,  =9
- Center control bonus (D4, E4, D5, E5)
- King centralization in endgames

# Minimax Algorithm on Chess env

Video Link:

- [Tap Here](#)

## Results and Performance analysis:

- Minimax performs decently in tactical positions
- Predictable due to fixed depth and deterministic evaluation
- Often results in draws or slow checkmates due to no learning/adaptation
- No pruning of unpromising moves leads to shallow depth in complex positions

## Key reasons for such performance:

- Shallow search depth limits long-term strategic planning - fixed depth search with depth = 3
- Evaluation function is simplistic and lacks positional understanding.
- Deterministic decisions cause repetitive and predictable gameplay.

# $\alpha\beta$ -pruning Algorithm on Chess env

## Environment Setup:

- Python-Chess with custom evaluator

## Minimax Search Logic:

- The algorithm recursively explores moves for both players, maximizing for White and minimizing for Black.
- If a branch can't possibly influence the final decision (i.e.  $\alpha \geq \beta$ ), it is skipped to save time.
- Once the search reaches the defined depth or a terminal state (checkmate/draw), it uses a static evaluation function to assign scores.

## Evaluation function based on:

- +1 for each  , +3 for   , +5 for  , +9 for 
- Checkmate =  $\pm 9999$
- 0 for draws / stalemates

# $\alpha\beta$ -pruning Algorithm on Chess env

Video Link:

- [Tap Here](#)

## Results and Performance analysis:

- Full Game Completion: AI plays until checkmate or draw in all test runs.
- Balanced Play: Makes sound material trades based on static evaluation.
- Consistent Results: White often wins due to the first-move advantage and symmetric depth search.

## Key reasons for such performance:

- Shallow Depth (3-ply): Limited foresight leads to missing long-term tactics.
- Simplistic Evaluation: No positional awareness (e.g., king safety, center control).
- Efficient Pruning: Alpha-Beta pruning significantly speeds up decision-making but still depends on move order quality.

# Final Comparison & Insights

| Feature             | Minimax                                                   | $\alpha\beta$ -pruning                                  |
|---------------------|-----------------------------------------------------------|---------------------------------------------------------|
| Search Strategy     | Full Minimax tree traversal                               | Minimax with branch pruning using alpha and beta bounds |
| Evaluation Function | Material count, central control, king proximity (endgame) | Material count only (simple, no positional factors)     |
| Code Structure      | minimax() and best_move() functions                       | alpha_beta() and get_best_move() methods                |
| Efficiency          | Slower – explores all nodes up to depth                   | Faster – prunes unnecessary branches                    |
| Heuristic Detail    | More nuanced with positional bonuses                      | Simple material-based scoring                           |